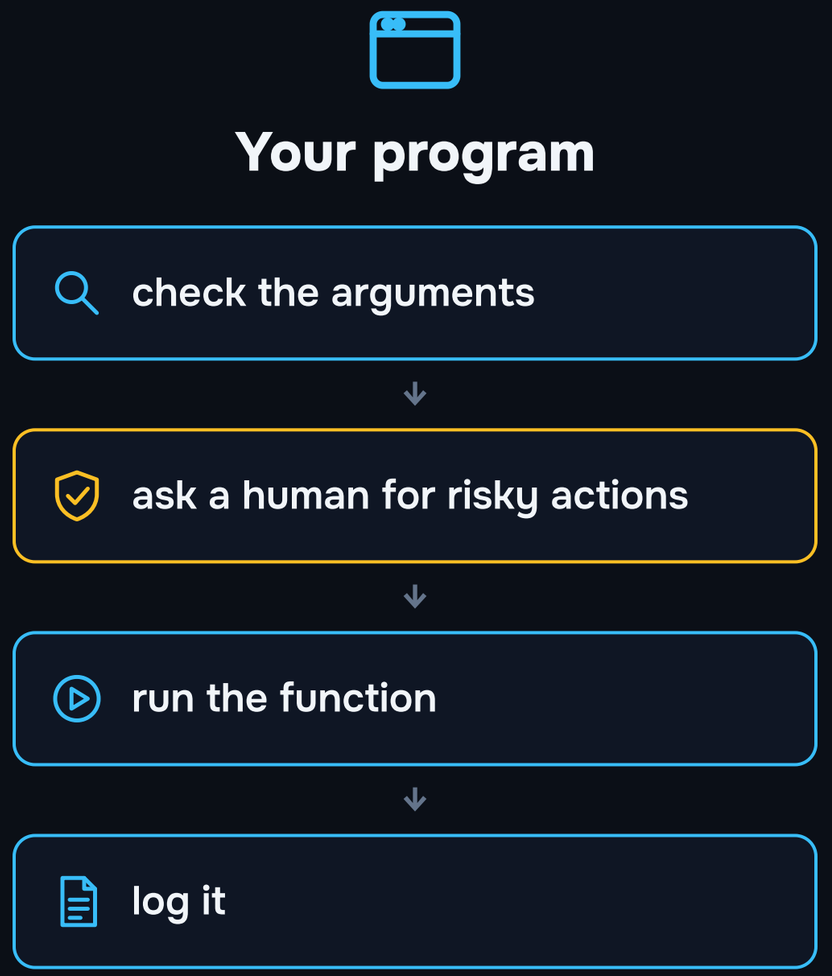
GENAI BASICS • LESSON 11 • MODULE 5: AGENTS, TOOLS AND MCP

Function calling and your first agent

How a model stops guessing and starts asking your system

Tools, function calling, the agent loop, risks. Google AI Studio, Claude, ChatGPT, local models. Data as of October 2026.



THE PROBLEM

The model does not know where your order is, but it will answer confidently

WITHOUT TOOLS

QUESTION

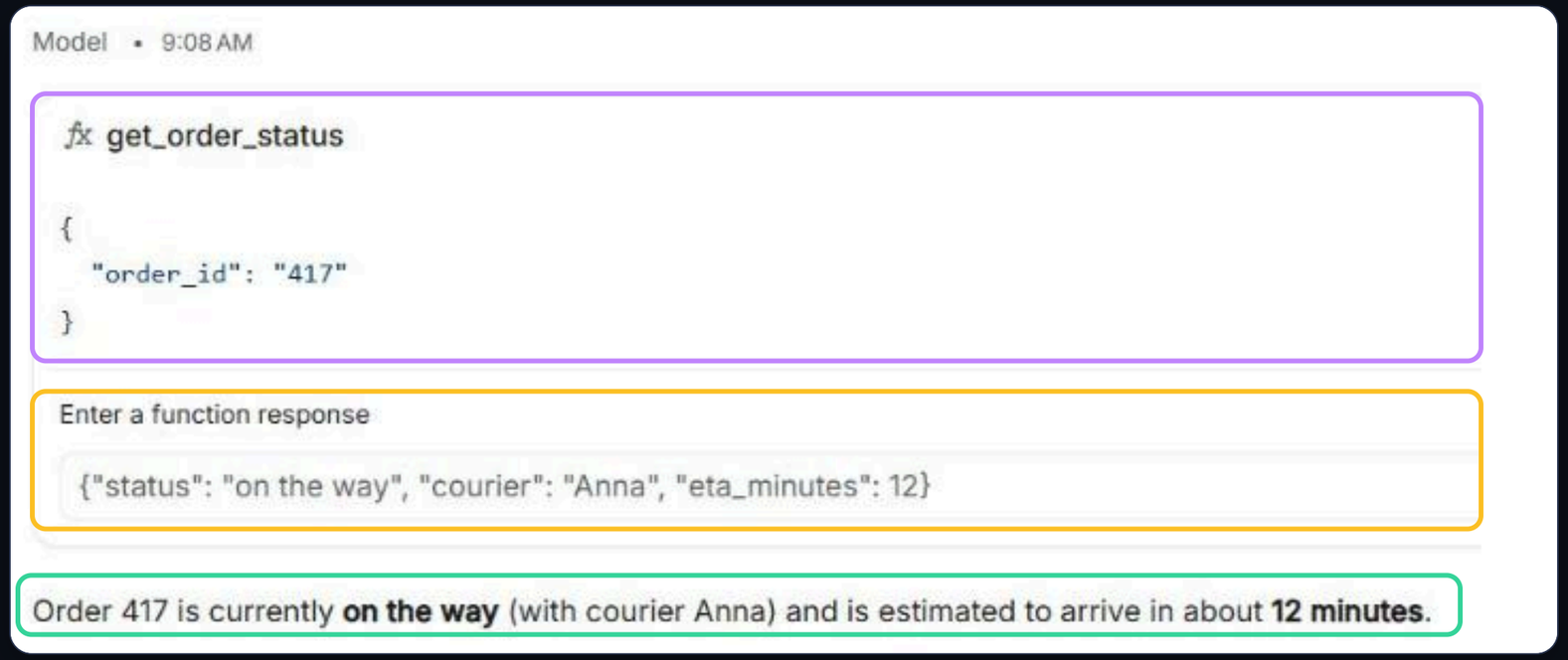
Where is order 417 right now?

MODEL ANSWER

“Your order 417 is on its way and will arrive tomorrow morning.”

Not a single lookup in the delivery system. Sounds convincing, but it is a guess.

WITH A TOOL · GOOGLE AI STUDIO, CAPTURED 16.10.2026

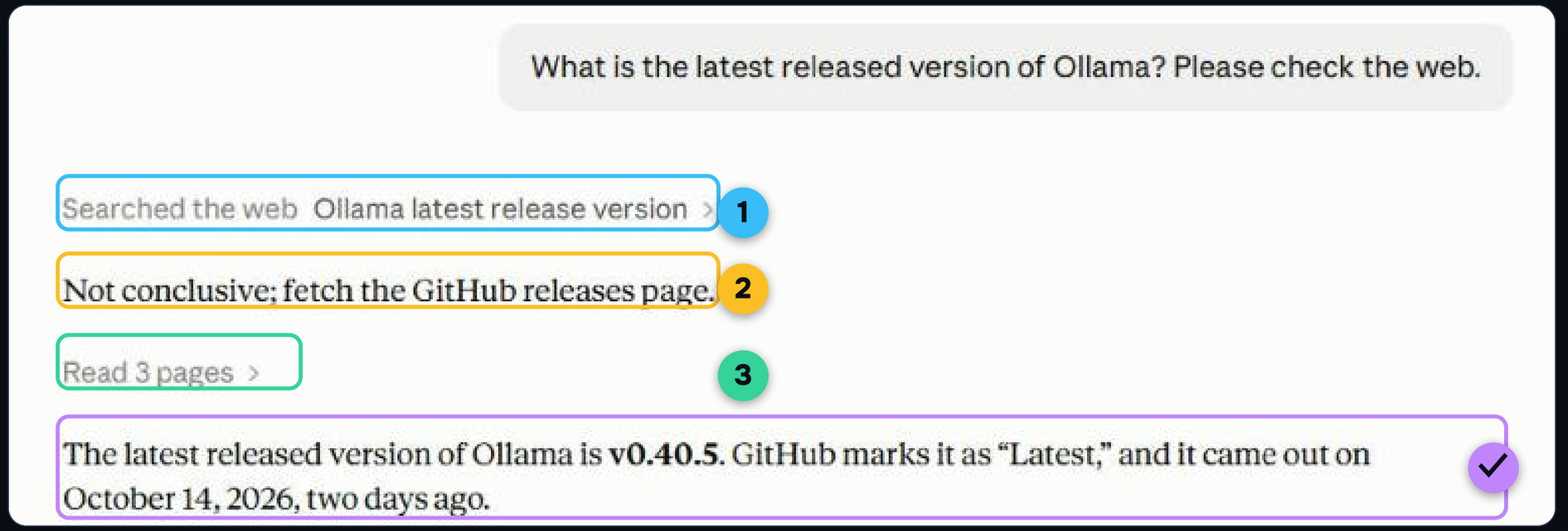


A **tool** is a function of your system the model can ask to call. The model asked for the order data (violet), got it (amber) and answered from it (green).

YOU HAVE SEEN THIS

Chat assistants already call tools: search and reading pages

CLAUDE · “LATEST OLLAMA VERSION?” · 16.10.2026



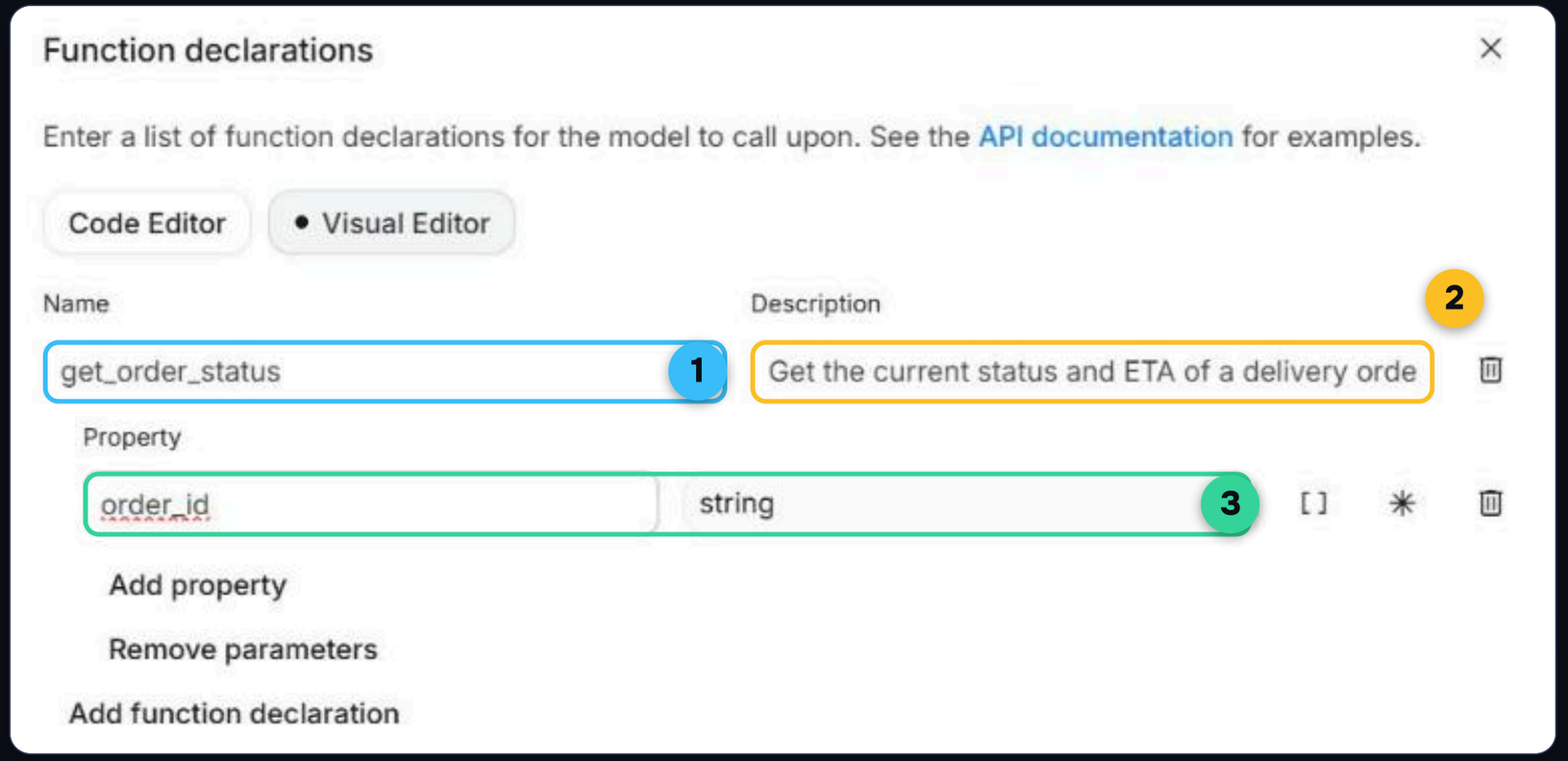
1. **Web search tool**: the model decided to search
2. **Judged the result**: “not conclusive, open the releases page”
3. **A second tool**: read 3 pages
4. **Answer** from what it found: v0.40.5 of 14 October

> Several steps in a row until the answer is ready: that is an **agent**.

A TOOL

A function declaration: a name, a description, parameters. That is all the model sees

AI STUDIO → RUN SETTINGS → TOOLS → FUNCTION CALLING → EDIT

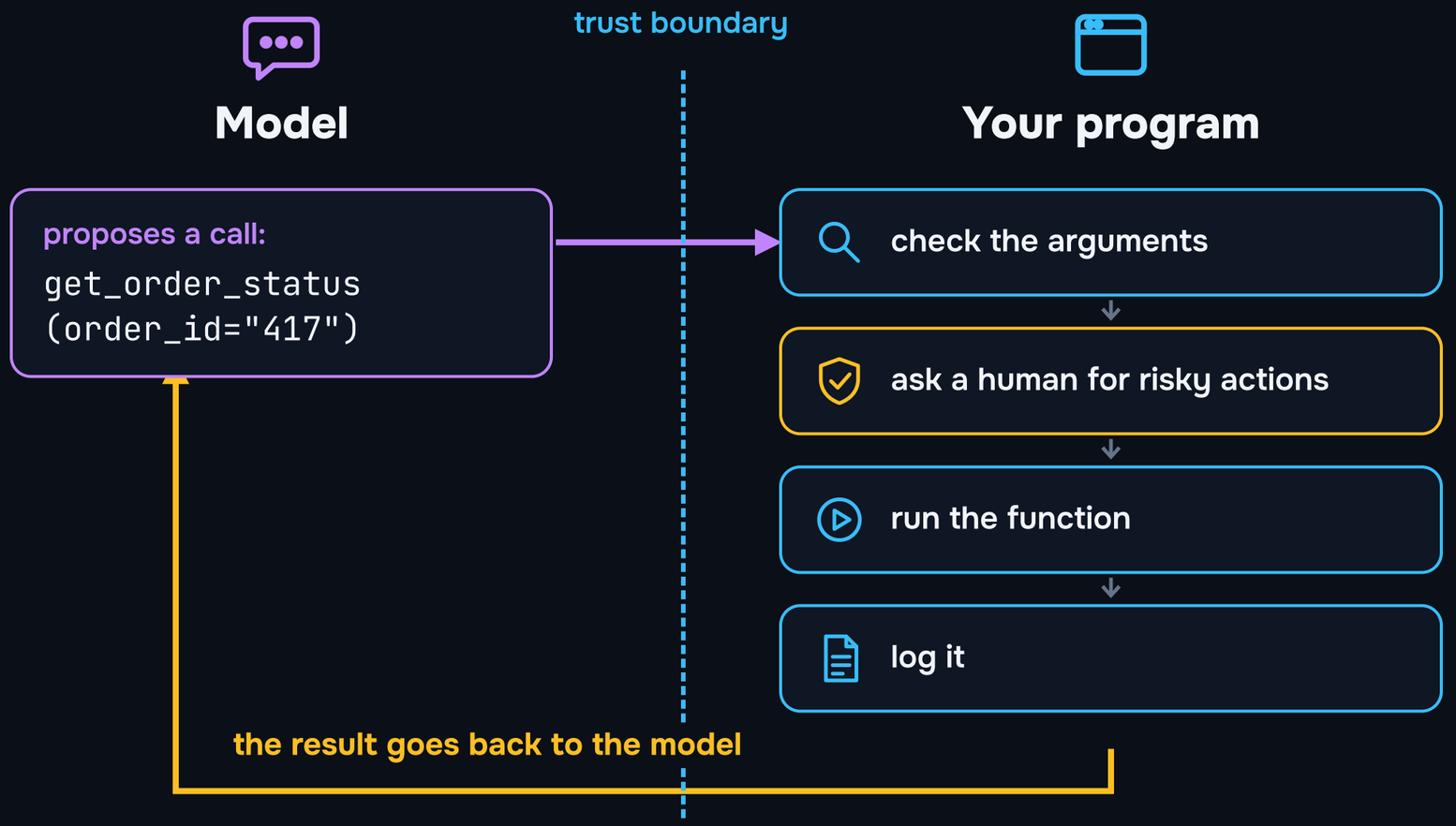


1. **Name**: `get_order_status`, short and unambiguous
2. **Description**: when to call this function. The model chooses by it
3. **Parameters**: what to pass, here the order number `order_id`

> The model does **not** see the function's code. It sees only this "card".

WHO DOES WHAT

The model only proposes a call. Your program runs it



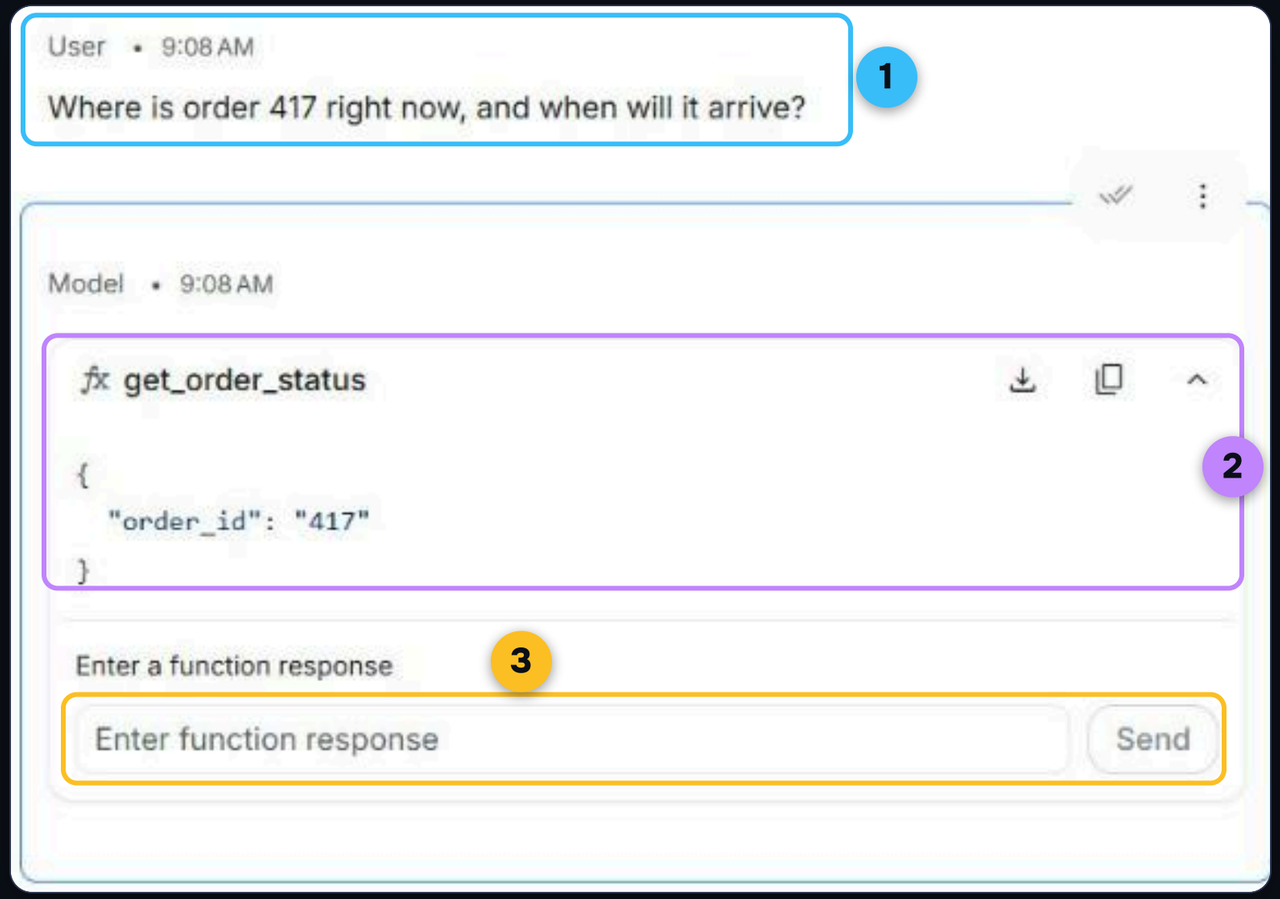
Trust boundary: on the left a model that can be wrong; on the right your program, which decides whether to run it.

Risky actions (refund money, send an email) — only after a human confirms.

DEMO • STEP 1

Demo: the model does not answer right away, it asks for a function call

AI STUDIO · GEMINI 3.8 FLASH · FREE · 16.10.2026



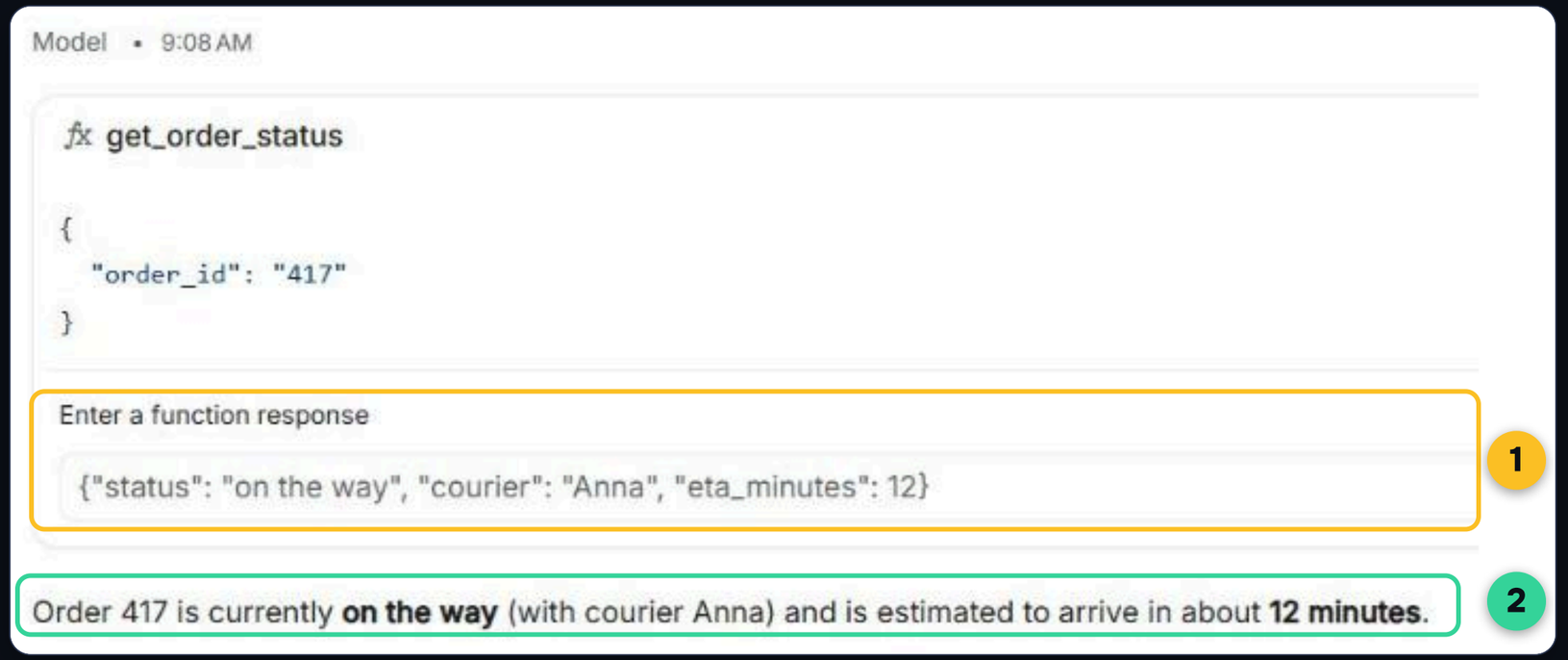
1. **The user's question**
2. **Function call**: name and argument `order_id: "417"`
3. **Function response field**: here you play the program

> The model worked out which function to call and what to pass: the number comes from the question.

DEMO • STEP 2

Return the result, and the model puts it into plain words

THE SAME CHAT AFTER THE FUNCTION RESPONSE



1 **Function response** (synthetic data)

```
{"status": "on the way",
"courier": "Anna", "eta_minutes":
12}
```

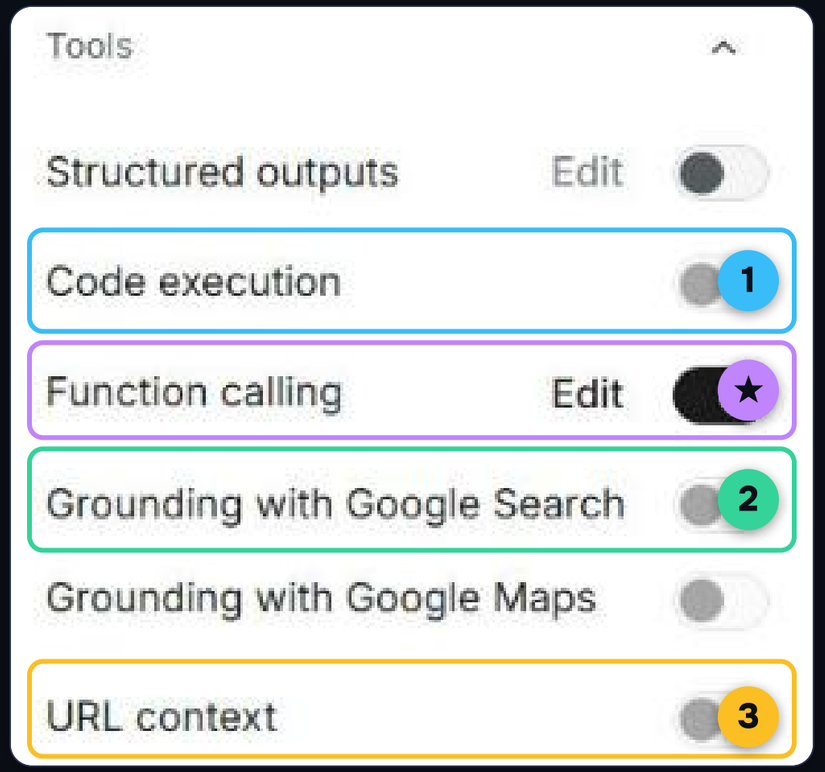
2 **Model answer** to the customer, based on that data

> Data from the system, words from the model. Each does what it is good at.

BUILT-IN TOOLS

Some tools come built in: code, search, reading pages

AI STUDIO · TOOLS



1. **Code execution**: compute, draw a chart
   Gemini · ChatGPT · Claude
2. **Web search**: fresh facts with links
   Gemini · ChatGPT · Claude
3. **Reading pages and files**: answer from a document
   Google: URL context · OpenAI: file search · Anthropic: web fetch

> **1–3 built-in**: the vendor runs them. ★ **Function calling**: your own function (orders, CRM, stock) is run by your program.

DESCRIPTION

The model picks a tool by its description. Write it like a task for an intern

✗ Weak

```
get_data
"Gets data."
```

Which data? When to call it? The model has to guess.

✓ Good

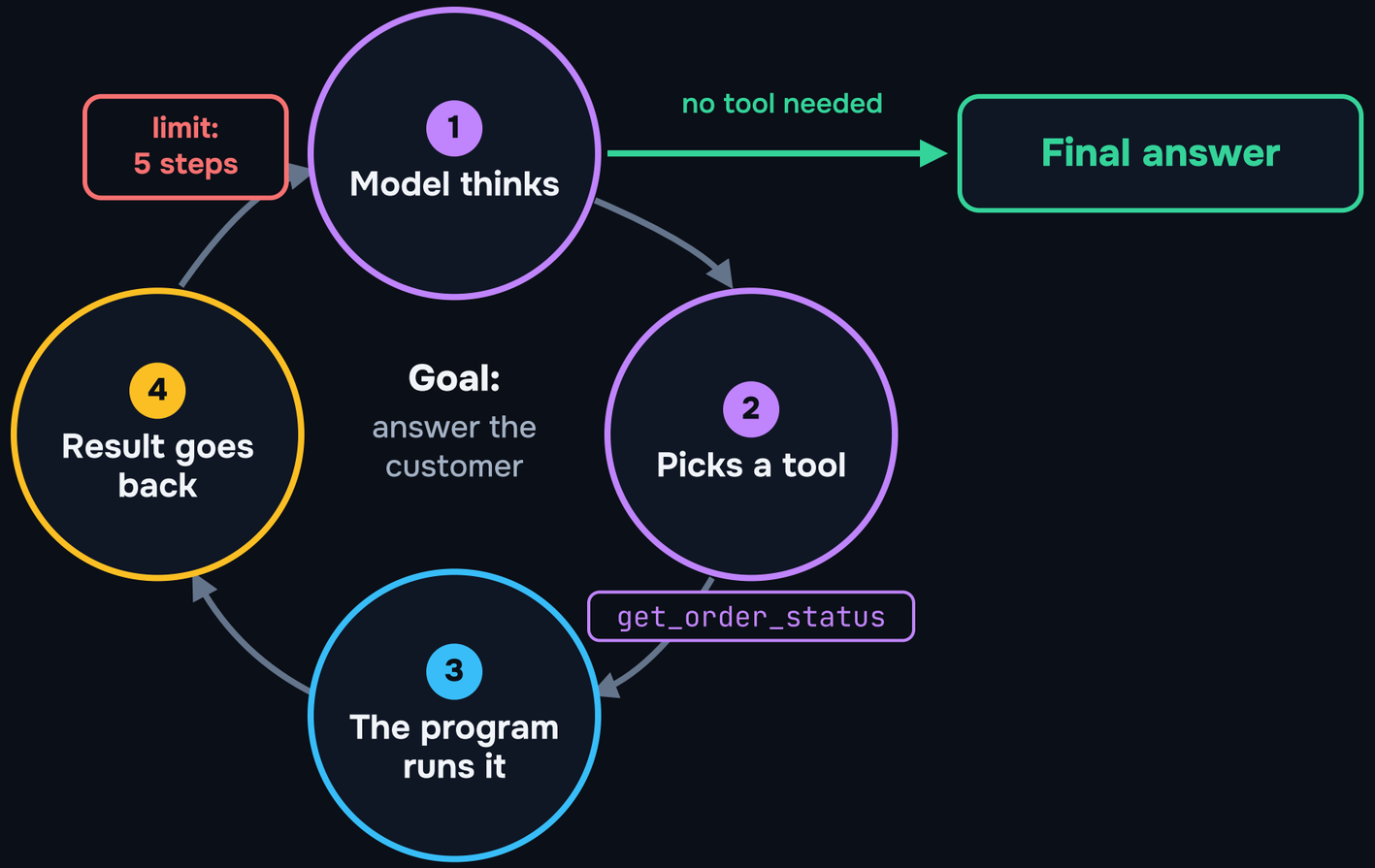
```
get_order_status
"Current status, courier and arrival time
of ONE delivery order by its number. Use
when the customer asks where an order is."
```

What it does, what for and when to call it. One tool, one job.

> If two tools sound alike (`get_order` and `find_order`), the model will mix them up. Rename or merge them.

AGENT

An agent = a model + tools + a loop until the task is done



Steps 1→4 repeat: the model may call several tools in a row, as Claude did on slide 3.

The loop ends when the model needs no more tools: it writes the **final answer**.

A **step limit** is a must: without it an agent can go round in circles and burn money.

CODE (REFERENCE)

In code the library runs the loop: a function plus its docstring

PYTHON, THE GOOGLE-GENAI LIBRARY (AS IN L03)

```python
from google import genai
from google.genai import types

def get_order_status(order_id: str) -> dict:
    """Current status and arrival time of one delivery order."""
    return {"status": "on the way", "courier": "Anna", "eta_minutes": 12}

client = genai.Client()          # key from GEMINI_API_KEY
r = client.models.generate_content(
    model="gemini-3.8-flash",
    contents="Where is order 417?",
    config=types.GenerateContentConfig(tools=[get_order_status]))
print(r.text)
```

The **docstring** is the tool description for the model.

tools=[...] — pass a plain Python function. The library calls it and hands the result back to the model.

OpenAI and Anthropic work the same way with different field names. Code is optional for the homework.

BRIDGE FROM L10

Local models can call functions too, but they make more mistakes

TOOL-CAPABLE IN OLLAMA AND LM STUDIO, OCTOBER 2026

- **Qwen3.5**
  0.8 to 122 billion parameters
  small sizes fit a laptop
- **Granite 4.2**
  3, 8 and 30 billion parameters
  small sizes fit a laptop
- **gpt-oss-20b**
  ≈ 16 GB memory
  needs a strong laptop or PC

The same idea locally: `localhost:1234` (LM Studio) and `localhost:11434` (Ollama) accept OpenAI-format tools. Use the `openai` library, as in L10, not google-genai.

A small model more often picks the wrong tool or argument format. Start with 1–2 simple functions and test.

HANDS-ON WITH AN AGENT

A coding agent can build your first agent from one prompt

PROMPT FOR A CODING AGENT: CODEX IN CHATGPT OR ANTIGRAVITY (FREE, WITH LIMITS), CLAUDE CODE (PAID)

```
Build a small Python support agent for a courier
service. Synthetic data only.
1. Two functions: get_order_status(order_id) and
create_return(order_id, reason), data from orders.json
(5 orders).
2. Run create_return only after I confirm in the console
(y/n).
3. Model: Gemini 3.8 Flash via google-genai, at most 5
calls per question.
4. Log every call: time, function, arguments, result.
5. Show 3 example dialogues and what is in the log.
```

The prompt already has every rule of the lesson: **narrow functions**, **confirmation** for the risky action, a **step limit**, a **log**.

You need your own Gemini API key (free tier with a daily limit: AI Studio → Get API key; enough for 3–4 test dialogues).

MARKET • OCTOBER 2026

Where agents live now: in chat apps, in developer kits, on your machine

← quicker to start, less control

more control, more work →

Ready-made agents in chat apps

the agent is inside the app: buttons and plans

- **ChatGPT Work**
  `agent and apps`
- **Claude**
  `search, code, connectors`
- **Gemini Tasks**
  `Pro/Ultra; not in the EEA`

Builders for developers

kits to build your own agent in code

- **OpenAI Agents SDK**
  `openai-agents`
- **Claude Agent SDK**
  `claude-agent-sdk`
- **Gemini API**
  `google-genai: automatic function calling`

Your own models

the agent runs entirely on your machine, as in L10

- **LM Studio · Ollama**
  `localhost`
- **Qwen3.5 · Granite 4.2**
  `Apache-2.0`
- **gpt-oss-20b**
  `Apache-2.0`

> Names change fast: ChatGPT agent was replaced by ChatGPT Work; OpenAI Agent Builder shuts down on 30.11.2026. Learn the mechanism, not the buttons.

BREAK

5-minute break. No tools

05:00

Next: risks, reliability, rules, an exercise and homework.

RISKS

Tools give the model hands. The main risk is someone else's text giving it orders

- **Prompt injection**
  a fetched page or email says "transfer the money" — the model may obey
- **Action without asking**
  the agent sent an email or created a refund on its own
- **Endless loop**
  the agent goes round in circles burning money
- **Too much access**
  the support bot can delete orders

Defences live in code

- ✓ least privilege for every tool
- ✓ a human confirms actions with consequences
- ✓ step and money limits
- ✓ a log of every call
- ✓ tool results are data, not commands

RELIABILITY

Even the best agents fail: on hard tasks, almost every second time

SOLVED IN ONE TRY (PASS^1), BEST RESULT ON EACH LEADERBOARD AS OF 16.10.2026

Support: retail, airline, telecom
τ²-bench test, best score



Banking
a different test, τ³-bench, best



0 50 100%

≈ 1 in 2

complex multi-step tasks the agent does not finish correctly

> That is why a human stays in the loop wherever a mistake costs money.

RULES

Four rules make an agent safer



THE MODEL PROPOSES

Narrow tools

one function, one job, a clear description

THE PROGRAM CHECKS

Checks in code

the program checks arguments and rights, not the model

A HUMAN CONFIRMS

Confirmation for risk

money, emails, deletion only after a “yes”

THE LOG RECORDS

Limits and a log

steps, budget, a record of every call

WHAT THE LOG LOOKS LIKE (EXAMPLE)

```
12:04:31  get_order_status(order_id="523")  → ok            step 2/5 · $0.002
12:04:33  create_return(order_id="523")     → waiting for an operator’s “yes”
```

> This is also the homework grading checklist.

EXERCISE

A good agent knows which tool to call and what to hand to a human

- **“Where is my order 523?”**
  → ?
- **“Refund me, the pizza was cold”**
  → ?
- **“What are your opening hours?”**
  → ?
- **“Delete my account and all orders”**
  → ?

The agent has two tools: `get_order_status` and `create_return`. Answers in the chat first, then we go through them together.

Hint: one question needs no tool, and one the agent must not carry out at all. Answers are in the speaker notes.

HOMEWORK

Homework: design an agent’s tools and test them in AI Studio

1. Pick a work task for an agent (support, warehouse, HR...)
2. Describe 2–3 functions: name, description, parameters
3. Repeat the AI Studio demo with your functions and synthetic data
4. Test 5 questions: which call the model proposed, and was it right
5. Describe the risks: where the confirmation goes, what limit, what is logged

```
genai-homeworks/L11/
├─ README.md   # task, conclusion
├─ tools.md    # 2-3 functions
├─ demo.png    # call and answer
└─ risks.md    # confirmation, limit
```

> Code optional: the slide 13 prompt.

Synthetic data only. Free: Gemini 3.8 Flash in AI Studio.

SUMMARY

The model proposes, the program decides, a human confirms

1 **The model does not know live data**

without a tool it guesses

2 **The model proposes, the program runs it**

the trust boundary lives in your code

3 **Agent = model + tools + loop**

with a step limit

4 **A human in the loop for risks**

money, emails, deletion

> **Next, L12:** MCP — one standard way to plug tools into any model, plus agent frameworks.